

Apunte 04 — Tablas de datos: pandas y el archivo parquet

Proyecto Froth · aprender Machine Learning construyendo

1 Qué conseguimos hoy

El primer botón del proyecto: **126 papers reales** sobre tu topic, guardados en `data/processed/papers.parquet`. Este apunte explica qué es esa “tabla” y por qué se guarda así.

Concepto: DataFrame (pandas)

pandas es LA librería de tablas en Python, y su objeto estrella es el **DataFrame**: una tabla con filas y columnas, como una hoja de Excel pero manejada con código. La nuestra tiene una fila por paper y estas columnas:

<code>id, doi</code>	identificadores únicos del paper
<code>title, year</code>	título y año
<code>citations</code>	cuántas veces lo han citado
<code>authors</code>	autores (texto “A; B; C”)
<code>abstract</code>	el resumen — la materia prima del ML
<code>query</code>	con qué búsqueda apareció (para depurar)

Con un DataFrame haces en una línea cosas que en Excel serían dolorosas:

- `df.drop_duplicates(subset="id")` quita papers repetidos;
- `df[df["abstract"].str.len() > 20]` filtra los que no tienen resumen;
- `df.sort_values("citations")` ordena por citas.

2 La limpieza importa más de lo que parece

`pull.py` no guarda lo que llega tal cual. Hace tres limpiezas:

1. **Deduplicar**: el mismo paper puede aparecer en varias búsquedas (“lepidolite flotation” y “lithium mica flotation” traen solapados).
2. **Filtrar sin-abstract**: sin resumen no hay significado que vectorizar.
3. **Reconstruir el abstract** (ver Apunte 02).

Ojo / error típico

En nuestros 126 papers se colaron algunos **off-topic** (p. ej. uno de telecomunicaciones). ¿Por qué? Colisión de siglas: buscamos “LCT” (un tipo de pegmatita) y también existe en otros campos. No lo arreglamos a mano: en las Fases 2–3 el propio ML los apartará, porque sus *vectores* quedarán lejos de los de flotación. Los datos reales siempre traen ruido; el sistema debe tolerarlo.

Concepto: parquet vs. CSV

Podíamos guardar la tabla como `.csv` (texto plano separado por comas). Elegimos **parquet** porque:

- **Comprime:** ocupa varias veces menos.
- **Guarda por columnas:** leer solo `year` y `citations` de 100 000 papers no obliga a leer los abstracts.
- **Conserva los tipos:** en CSV todo vuelve como texto y hay que re-adivinar qué era número; parquet recuerda que `year` es un entero.

Es el formato estándar en data science moderno. La librería `pyarrow` es quien sabe leerlo/escribirlo; pandas la usa por debajo con `df.to_parquet()` y `pd.read_parquet()`.

Pruébalo tú (teclado en mano)

Mira tu botón con tus propias manos. En PowerShell, dentro de la carpeta del proyecto, abre un Python interactivo con `.venv\Scripts\python.exe` y escribe línea a línea:

1. `import pandas as pd`
2. `df = pd.read_parquet("data/processed/papers.parquet")`
3. `len(df)` (¿cuántos papers?)
4. `df.columns` (las columnas)
5. `df.sort_values("citations", ascending=False).head(3)[["title", "year"]]`
6. `exit()` para salir.

Acabas de hacer tu primer análisis de datos en pandas.

En una frase

Un DataFrame es una tabla programable; limpiamos duplicados y sin-abstract; y parquet es el formato comprimido, por columnas y con tipos que usa el data science serio.